Split Federated Learning for Resource-Constrained Edge Computing Networks

Xianghe Wang, Shaoyang Song, Zekai Zhang, Xiangwang Hou, *Member, IEEE*, Zhiying Li, Tianyu Xing, and Xiao-Ping Zhang, *Fellow, IEEE*

***Abstract*— Compute First Networking (CFN) integrates distributed computing resources across cloud, edge, and device layers to meet diverse computing demands. Within this context, Split Federated Learning (SFL) has emerged as an effective approach to reduce device workload and enhance privacy by integrating model splitting into federated training. However, deploying SFL in edge computing networks is challenging due to device heterogeneity, unreliable channels, and the trade-off between convergence and resource usage. This paper formulates a joint optimization problem that considers split layer selection, computing resource allocation, and transmission power control to minimize the total training latency required to reach a target model accuracy. We propose an efficient Block Coordinate Descent (BCD)-based algorithm to solve this problem. Experimental results demonstrate that the proposed framework significantly reduces the total training latency and adapts effectively to varying wireless conditions.**

***Index Terms*— Edge computing networks, split federated learning, model splitting, transmission power control, computing resources allocation.**

I. Introduction

In THE forthcoming era of Network 2030, the convergence of networking and computing has fostered the paradigm of Compute First Networking (CFN), which envisions a unified infrastructure that seamlessly integrates cloud, edge, and device-level computing resources [1]. As an essential component of CFN, edge computing networks bring computation closer to data sources and end users, thereby reducing latency and enhancing service responsiveness.

However, applying centralized model training in such networks is often infeasible. The rapid growth of intelligent applications generates massive amounts of distributed data, and transmitting this data to a central server incurs prohibitive communication overhead and potential privacy concerns [2]. These limitations highlight the necessity of decentralized learning approaches tailored for edge environments. As a result, distributed machine learning (DML) has become a key enabler for edge computing networks, allowing devices to collaboratively process data and train models.

Among DML approaches, Federated Learning (FL) has gained prominence for its privacy-preserving nature [3]. In FL, devices download a global model, perform local updates using private data, and send model parameters—not raw data—back to a central server for aggregation. However, FL assumes that each device can process the entire model, which becomes infeasible for modern DNNs. For example, storing models like AlexNet [4] or VGG16 [5] requires hundreds of megabytes of memory, which exceeds the capabilities of many edge devices. Additionally, large models incur significant communication overhead in FL due to repeated uploads and downloads of full model parameters [6].

To alleviate these limitations, Split Learning (SL) has been proposed [7], in which a model is partitioned between the client and the server to balance computation. However, SL is typically designed for sequential client-server execution, which can lead to excessive training delays. To further reduce latency and communication costs, Split Federated Learning (SFL) integrates the collaborative structure of FL with the partitioned model architecture of SL [8], allowing parallelized training and synchronized aggregation across devices. SFL has demonstrated improved communication efficiency and privacy protection [9], [10].

Despite its promise, deploying SFL efficiently in resource-constrained edge networks remains challenging. Issues such as device heterogeneity, unreliable wireless channels, and the trade-off between convergence speed and resource usage complicate practical implementations. These challenges highlight the need to develop a resource-efficient and adaptive SFL framework for edge computing networks operating under unreliable wireless conditions.

A. Related Works

Since Thapa et al. [8] first introduced the concept of SFL, a number of studies have emerged that explore SFL from various perspectives. Zhang et al. [11] proposed SplitAVG

Received 14 March 2025; revised 15 June 2025 and 28 August 2025; accepted 18 October 2025. Date of publication 28 October 2025; date of current version 8 December 2025. This work was supported in part by the Shenzhen Ubiquitous Data Enabling Key Lab under Grant ZDSYS20220527171406015 and in part by the Tsinghua Shenzhen International Graduate School-Shenzhen Pengrui Endowed Professorship Scheme of Shenzhen Pengrui Foundation. *(Corresponding author: Xiao-Ping Zhang.)*

Xianghe Wang and Xiangwang Hou are with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China (e-mail: wangxh24@mails.tsinghua.edu.cn; xiangwanghou@163.com).

Shaoyang Song is with the School of Information and Electronics, Beijing Institute of Technology (BIT), Beijing 100081, China, and also with the Yangtze Delta Region Academy, BIT (Jiaxing), Jiaxing 314000, China (e-mail: 3220230961@bit.edu.cn).

Zekai Zhang, Tianyu Xing, and Xiao-Ping Zhang are with the Shenzhen Key Laboratory of Ubiquitous Data Enabling, Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China (e-mail: zhangzekai_2000@163.com; xingtianyu21@163.com; xpzhang@ieee.org).

Zhiying Li is with the School of Instrumentation and Opto-Electronic Engineering, Beihang University, Beijing 100191, China (e-mail: lizhiying@buaa.edu.cn).

Digital Object Identifier 10.1109/TCE.2025.3626460

1558-4127 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

based on the basic framework of SFL for medical imaging, and validated the performance of SplitAVG as well as its robustness to data heterogeneity across multiple tasks. According to the general setup of SFL, different participants in SFL transmit data labels, which can also lead to a certain degree of privacy leakage. Yang et al. [12] achieved privacy protection for model parameters, data inputs, outputs, and labels by dividing the model network into three parts hosted by different parties, enabling SFL to offer improved privacy protection. Other studies have employed methods such as differential privacy [13] and homomorphic encryption [14] as defense mechanisms for SFL to counteract attacks.

Despite the advantages of SFL in terms of computational load balancing and privacy preservation, it still inherits critical challenges from both FL and SL frameworks, especially regarding device heterogeneity and convergence latency. One of the most crucial factors is the selection of the split layer, which directly affects the computational burden on clients and servers as well as the communication overhead, thereby influencing the overall training latency [15]. Several studies have proposed methods for optimizing the split layer selection. In [16], a regression model was designed to predict latency and energy consumption across candidate layers, enabling the system to select the optimal split layer. Wu et al. [17] mitigated the effects of device heterogeneity and dynamic network conditions on training by optimizing split layer selection. Lin et al. [18] further proposed an adaptive split-point selection scheme that enables personalized model partitioning for each device to minimize training latency. In addition, a few studies investigated model partitioning under inference scenarios by constructing predictive models for latency and energy to determine optimal split layers [16], [19]. Under dynamic wireless conditions, [20] further explored optimal DNN split locations for delay minimization. Most notably, [21] investigated optimal multi-point splitting across multiple computing nodes, offering a practical path toward heterogeneous split strategies that align with device capabilities.

A wide spectrum of research has been dedicated to balancing training efficiency and resource usage of FL from various perspectives. Lin et al. [22] investigated SFL for heterogeneous edge devices, quantified how batch size and model splitting affect convergence, and based on this analysis derived the minimum number of training rounds needed to achieve a target model accuracy, upon which they formulated a latency-minimization problem and proposed the HASFL framework to adaptively control batch sizes and model partitioning across heterogeneous networks. Li et al. [23] proposed a multi-bit over-the-air computation (M-AirComp) scheme to efficiently aggregate local model updates and jointly optimized transmission probability and local computing to minimize the long-term energy consumption of mobile devices in FL. Tang et al. [24] evaluated the effect of UAV deployment on sensing performance and formulated a long-term training latency minimization problem, which was addressed using an alternating optimization-based BBPO (bandwidth, batch size, and position optimization) scheme for joint UAV deployment and resource allocation under non-ideal sensing conditions.

Recent studies have also begun to consider FL under non-ideal wireless environments, focusing on communication reliability and resource management. In [25], the impact of packet error rates on the convergence behavior of FedAvg was analyzed, and a joint strategy for client selection and resource allocation was proposed to enhance convergence speed. The work in [26] introduced the concept of transmission outage (TO) probability into the local model aggregation process, thereby redesigning the averaging protocol to better accommodate unreliable links. In [27], the waveform superposition properties of broadband channels were leveraged to reduce transmission latency, while the effects of channel fading and imperfect channel state information on FL convergence were systematically investigated. Collectively, these efforts underscore the necessity of adapting learning framework to the characteristics of dynamic and error-prone wireless communication—a key consideration in the deployment of edge computing networks.

B. Contributions

Based on the above analysis, this paper proposes a resource-saving SFL scheme for edge computing networks, which integrates split layer selection, computing resource allocation, and transmission power control, taking into account device heterogeneity and non-ideal wireless environments. The main contributions are as follows:

- We propose an efficient SFL framework for resource-constrained edge computing networks, integrating split layer selection, computing resource allocation, and power control. Additionally, by analyzing the convergence of SFL in non-ideal wireless environments and characterizing the delay model, we formulate a joint optimization problem to minimize training delay while ensuring model convergence.
- Since the problem we formulate is non-convex, we decompose it into three sub-problems: split layer selection, computing resource allocation, and transmit power control. We solve these sub-problems using traversing method, convex optimization, and Bayesian optimization, respectively. Then, we employ the alternating optimization method to obtain an effective solution to the original problem.
- We evaluate the performance of our proposed scheme using various public datasets and under different data distributions. Results show that, compared with baselines, our proposed scheme achieves the lowest delay while maintaining satisfactory convergence performance and exhibits strong adaptability across diverse environments.

The remainder of this paper is structured as follows. Section II outlines the system model. Section III delves into the convergence analysis of SFL. The optimization problem is formulated and the solution algorithm is introduced in Section IV. Simulation results are then discussed in Section V, followed by the conclusion in Section VI.

II. System Model

The proposed SFL framework for resource-constrained edge computing networks, as shown in Fig. 1, consists of an

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

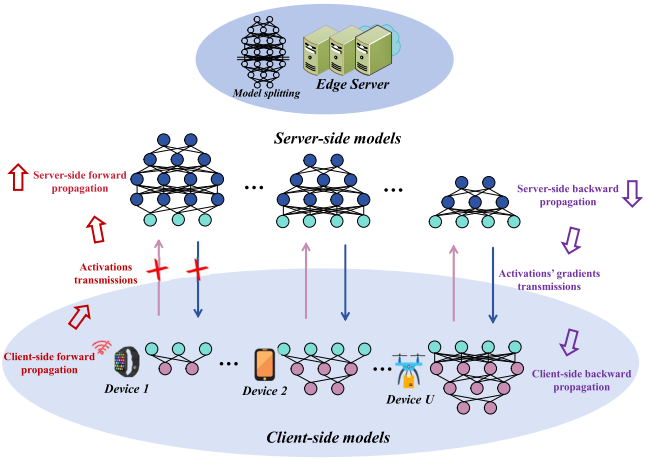


Fig. 1. The illustration of SFL over resource-constrained edge computing networks.

edge server and a set of U edge devices, denoted as $\mathcal{U} = \{1, 2, \ldots, u, \ldots, U\}$, and is designed to collaboratively train a ML model. Each device u holds a local dataset $\mathcal{D}_u$, represented by $\mathcal{D}_u = \{(\boldsymbol{x}_{u,1}, \boldsymbol{p}_{u,1}), (\boldsymbol{x}_{u,2}, \boldsymbol{p}_{u,2}), \ldots, (\boldsymbol{x}_{u,N_u}, \boldsymbol{p}_{u,N_u})\}$, where N_u represents the number of data samples, $\boldsymbol{x}_{u,i}$ and $\boldsymbol{p}_{u,i}$ denote the data sample and one-hot label, respectively. Considering the limited computational resources and battery power of edge devices, the L-layered complex machine learning model $\boldsymbol{w}$ is divided into two parts: the client-side model $\boldsymbol{w}_{c,u}$ and the server-side model $\boldsymbol{w}_{s,u}$. The client-side model $\boldsymbol{w}_{c,u}$ is deployed on the edge devices, while the server-side model $\boldsymbol{w}_{s,u}$ is deployed on the edge server. Specifically, $\boldsymbol{w}_{c,u}$ includes the first L_u layers of model $\boldsymbol{w}$, and $\boldsymbol{w}_{s,u}$ contains the layers from $L_u + 1$ to L, where L_u denotes the split layer for device u. Meanwhile, we use $\mathcal{L}_u = (x_u, y_u, z_u)$ and $\mathcal{L}_s = (x_s, y_s, z_s)$ to represent the 3-dimensional coordinates of devices u and the edge server, respectively.

A. Split Federated Learning Model

The training process aims to minimize the global loss function $\mathcal{F}(\boldsymbol{w})$, which is defined as

$$\mathcal{F}(\boldsymbol{w}) = \frac{1}{U} \sum_{u=1}^{U} \mathcal{F}_u(\boldsymbol{w}), \tag{1}$$

where $\mathcal{F}_u(\boldsymbol{w}) \triangleq \mathbb{E}_{\xi_u \sim \mathcal{D}_u}\left[f_u(\boldsymbol{w}; \xi_u)\right]$ is the local loss function for device u, while ξ_u represents the mini-batch data randomly sampled from the dataset $\mathcal{D}_u$.

As illustrated in Fig. 1, the SFL workflow can be divided into five phases: 1) Model splitting; 2) Model distribution; 3) Model training; 4) Model collection; and 5) Model aggregation. The model splitting strategy is outlined in Section IV. In the following, we provide a detailed description of the model training and model aggregation phases.

Model Training: In training round n, each client and the server perform $\mathcal{E} = \frac{|\mathcal{D}_u|}{\delta_u} \cdot E$ iterations, where E is the number of local training epochs and δ_u is the mini-batch size. For each local iteration $e \leq \mathcal{E}$, the following procedure is executed.

1) Client-Side Forward Propagation: Each device u performs forward propagation in parallel. The activation values of the l_c-th layer ($l_c \in \{1, \ldots, L_u\}$) at e-th iteration ($e \in \{1, \cdots, \mathcal{E}\}$) can be defined as [28]:

$$\mathbf{A}_{c,u}^{n,e}(l_c) = \delta\left(\boldsymbol{w}_{c,u}^{n,e-1}(l_c)\, \mathbf{A}_{c,u}^{n,e}(l_c - 1) + b_{c,u}^{n,e}(l_c)\right), \tag{2}$$

where δ denotes the activation function, with $\boldsymbol{w}_{c,u}^{n,e-1}(l_c)$ and $b_{c,u}^{n,e}(l_c)$ being the weights and biases, respectively.

2) Activations Transmissions: After completing the client-side forward propagation, each device transmits the smashed data, including the activation values $\mathbf{A}_{c,u}^{n,e}(L_u)$ and their corresponding labels $\mathcal{H}_u$, to the server for the server-side training process, which can be denoted as

$$\boldsymbol{S}_u^{n,e} = \left(\mathbf{A}_{c,u}^{n,e}(L_u), \mathcal{H}_u\right). \tag{3}$$

3) Server-Side Forward Propagation: After receiving the smashed data, the edge server will perform forward propagation through the remaining layers, such as the activation of layer l_s ($l_s \in \{L_u + 1, L_u + 2, \ldots, L\}$) which can be calculated as

$$\mathbf{A}_{s,u}^{n,e}(l_s) = \delta\left(\boldsymbol{w}_{s,u}^{n,e-1}(l_s)\, \mathbf{A}_{s,u}^{n,e}(l_s - 1) + b_{s,u}^{n,e}(l_s)\right), \tag{4}$$

where $\mathbf{A}_{s,u}^{n,e}(L_u) = \mathbf{A}_{c,u}^{n,e}(L_u)$.

4) Server-Side Backward Propagation: The server-side models compute the loss function value based on the predicted values and labels, and then perform backpropagation to obtain the gradients $\mathbf{g}_u^{n,e}\left(\boldsymbol{w}_{s,u}^{n,e-1}\right)$ for each server-side model:

$$\mathbf{g}_u^{n,e}\left(\boldsymbol{w}_{s,u}^{n,e-1}\right) = \nabla\mathcal{F}_u\left(\boldsymbol{w}_{s,u}^{n,e-1}\right) = \frac{\partial\mathcal{F}_u(\boldsymbol{w}_u^{n,e-1})}{\partial\boldsymbol{w}_{s,u}^{n,e-1}}, \tag{5}$$

Thus, the server-side model is updated by

$$\boldsymbol{w}_{s,u}^{n,e} = \boldsymbol{w}_{s,u}^{n,e-1} - \gamma\mathbf{g}_u^{n,e}\left(\boldsymbol{w}_{s,u}^{n,e-1}\right), \tag{6}$$

where γ is a constant parameter denoting the learning rate.

5) Activations' Gradients Transmissions: The server transfers the smashed data's gradient $\frac{\partial\mathcal{F}_u(\boldsymbol{w}_u^{n,e-1})}{\partial\mathbf{A}_{c,u}^{n,e}(L_u)}$ to device u for backward propagation.

6) Client-Side Backward Propagation: After obtaining gradients $\frac{\partial\mathcal{F}_u(\boldsymbol{w}_u^{n,e-1})}{\partial\mathbf{A}_{c,u}^{n}(L_u)}$ transmitted from the edge server, each device computes the gradient $\mathbf{g}_u^{n,e}\left(\boldsymbol{w}_{c,u}^{n,e-1}\right)$:

$$\begin{aligned}\mathbf{g}_u^{n,e}\left(\boldsymbol{w}_{c,u}^{n,e-1}\right) &= \nabla\mathcal{F}_u\left(\boldsymbol{w}_{c,u}^{n,e-1}\right) \\ &= \frac{\partial\mathcal{F}_u(\boldsymbol{w}_u^{n,e-1})}{\partial\mathbf{A}_{c,u}^{n,e}(L_u)} \cdot \frac{\partial\mathbf{A}_{c,u}^{n,e}(L_u)}{\partial\boldsymbol{w}_{c,u}^{n,e-1}}\end{aligned} \tag{7}$$

and the client-side model is updated by

$$\boldsymbol{w}_{c,u}^{n,e} = \boldsymbol{w}_{c,u}^{n,e-1} - \gamma\mathbf{g}_u^{n,e}\left(\boldsymbol{w}_{c,u}^{n,e-1}\right) \tag{8}$$

Model Aggregation: After $\mathcal{E}$ iterations of model training (i.e., at the end of round n), the devices send the client-side model $\boldsymbol{w}_{c,u}^{n,\mathcal{E}}$ to the server for model aggregation. The server-side model $\boldsymbol{w}_{s,u}^{n,\mathcal{E}}$ is combined with the client side model $\boldsymbol{w}_{c,u}^{n,\mathcal{E}}$ to

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

construct the final local model $\boldsymbol{w}_u^{n,\mathcal{E}} = \boldsymbol{w}_{s,u}^{n,\mathcal{E}} \uplus \boldsymbol{w}_{c,u}^{n,\mathcal{E}}$. And the global model can be aggregated by

$$\boldsymbol{w}_n = \frac{1}{U}\sum_{u=1}^{U} \boldsymbol{w}_u^{n,\mathcal{E}}. \tag{9}$$

B. Wireless Communication Model

We assume that the orthogonal frequency division multiple access (OFDM) is employed in the uplink transmission. The data rate of uplink transmission from the device u to the server is determined by [29]

$$R_u^{\mathrm{UL}} = W \log_2\left(1 + \frac{p_u h_u}{I_u + W N_0}\right), \tag{10}$$

where W denotes the bandwidth allocated to each device u, which is assumed to be equal across all devices. Additionally, let p_u denote the transmission power, N_0 the noise power spectral density, and I_u the interference. The channel gain of device u is denoted by h_u, defined as

$$h_u = \varsigma_u (d_u)^{-2}, \tag{11}$$

where ς_u characterizes the Rayleigh fading [30], and $d_u = \|\mathcal{L}_u - \mathcal{L}_s\|_2$ denotes the distance between the server and the device u.

Similarly, the downlink transmission rate from the server to each device is expressed as

$$R_u^{\mathrm{DL}} = W_{\mathrm{BS}} \log_2\left(1 + \frac{p_B h_u}{I_u + W_{\mathrm{BS}} N_0}\right), \tag{12}$$

where p_B denotes the transmission power of server, and W_{BS} denotes the bandwidth that the server used to broadcast the global model to each device.

In wireless communication, errors are an unavoidable reality, which is contrasting with ideal settings. We consider the transmission from each device as a single packet and implement Cyclic Redundancy Check (CRC) to ascertain if there are any transmission errors. The transmission outage probability from device u to the server can be denoted as

$$q_u(p_u) = 1 - \exp\left(-\frac{\Upsilon(I_u + W N_0)}{p_u h_u}\right), \tag{13}$$

where Υ represents the waterfall threshold [31]. Assign α_u^n to indicate whether the device u can successfully communicate with the server at n-th round, which is given by

$$\alpha_u^n = \begin{cases} 1, & \text{if } 1 - q_u(p_u), \\ 0, & \text{if } q_u(p_u). \end{cases} \tag{14}$$

The value of $\alpha_u^n = 0$ indicates that device u fails to participate in training during the n-th round due to communication failure. Accordingly, based on partial participation aggregation, Eq. (9) can be revised as

$$\boldsymbol{w}_n = \frac{\sum_{u=1}^{U} \alpha_u^n \boldsymbol{w}_u^{n,\mathcal{E}}}{\sum_{u=1}^{U} \alpha_u^n}. \tag{15}$$

C. Training Delay Model

Given that the training rounds are independent, we focus on a single round for analysis. Based on the above SFL and wireless communication models, a detailed analysis of the training delay is conducted as follows [32].

1) Client-Side Model Transmission Delay: The client-side model transmission includes model distribution process from the server and local model update process for model aggregation. Let $\zeta(L_u)$ indicate the size (in bits) of client-side model. Thus, the delay of model distribution and model update can be denoted as

$$\tau_{\mathrm{dm},u}^n(L_u, p_u) = \alpha_u^n \cdot \frac{\zeta_u(L_u)}{R_u^{\mathrm{DL}}} \tag{16}$$

and

$$\tau_{\mathrm{um},u}^n(L_u, p_u) = \alpha_u^n \cdot \frac{\zeta_u(L_u)}{R_u^{\mathrm{UL}}}. \tag{17}$$

2) Client-Side Model Forward Propagation Delay: Let $\mathbf{C}_c^F(L_u)$ denote the computation workload per data sample during the forward propagation of device u. We consider each device u training for E epochs on its local dataset $\mathcal{D}_u$ (where the size of $\mathcal{D}_u$ is denoted as D_u) per round, thereby the delay can be represented as

$$\tau_{\mathrm{fp},u}^n(L_u, p_u) = \alpha_u^n \cdot \frac{D_u \mathbf{C}_{c,u}^F(L_u) \cdot E}{f_u \varpi}, \tag{18}$$

where f_u is the CPU frequency of device u and ϖ represents the computing intensity [17].

3) Smashed Data Transmission Delay: The transmission delay to upload smashed data of device u can be given by

$$\tau_{\mathrm{sd},u}^n(L_u, p_u) = \alpha_u^n \cdot \frac{D_u \psi_{a,u}(L_u) \cdot E}{R_u^{\mathrm{UL}}}, \tag{19}$$

where $\psi_{a,u}(L_u)$ denotes the size of smashed data.

4) Server-Side Model Execution Delay: The server-side model execution involves both forward and backward propagation. Let $\mathbf{C}_s(L_u)$ denote the per-sample computation workload on the server side; the corresponding delay is then given by

$$\tau_{\mathrm{s},u}^n(L_u, p_u, f_{s,u}) = \alpha_u^n \cdot \frac{D_u \mathbf{C}_{s,u}(L_u) \cdot E}{f_{s,u} \varpi}, \tag{20}$$

where $f_{s,u}$ denotes the CPU frequency allocated by the server to the device u.

5) Smashed Data's Gradient Transmission Delay: Let $\psi_{g,u}(L_u)$ represent the size (in bits) of smashed data's gradient and the delay can be given by

$$\tau_{\mathrm{g},u}^n(L_u, p_u) = \alpha_u^n \cdot \frac{D_u \psi_{g,u}(L_u) \cdot E}{R_u^{\mathrm{DL}}}. \tag{21}$$

6) Client-Side Model Update Delay: The client-side model update refers to the process in which device u updates its local model parameters via backward propagation. Let $\mathbf{C}_c^B(L_u)$ denote the per-sample computation workload during backward propagation at device u, the corresponding delay is given by

$$\tau_{\mathrm{bp},u}^n(L_u, p_u) = \alpha_u^n \cdot \frac{D_u \mathbf{C}_{c,u}^B(L_u) \cdot E}{f_u \varpi}. \tag{22}$$

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

Based on the analysis of Eq. (16) to Eq. (22), we can derive the overall delay in a single round for each device u:

$$\tau_u^n\left(L_u, p_u, f_{s,u}\right) = \tau_{\mathrm{dm},u}^n + \tau_{\mathrm{um},u}^n + \tau_{\mathrm{fp},u}^n + \tau_{\mathrm{sd},u}^n + \tau_{\mathrm{s},u}^n + \tau_{\mathrm{g},u}^n + \tau_{\mathrm{bp},u}^n. \tag{23}$$

We consider the proposed SFL framework is synchronous on the server side, i.e., aggregation is performed only after receiving local models from every participating devices. Therefore, the overall time delay in training round n can be expressed as

$$\tau^n\left(\{L_u\}, \{f_{s,u}\}, \{p_u\}\right) = \max_{u\in\mathcal{U}}\left(\tau_{dm,u}^n + \tau_{um,u}^n + \tau_{fp,u}^n + \tau_{sd,u}^n + \tau_{s,u}^n + \tau_{g,u}^n + \tau_{bp,u}^n\right). \tag{24}$$

III. Convergence Analysis

Ahead of probing the convergence analysis, some widely accepted assumptions are introduced as below [33] [34]:

Assumption 1: Each local function $\nabla\mathcal{F}_u(\boldsymbol{w})$ is uniformly β-smooth with respect to any $\boldsymbol{w}$ and $\boldsymbol{w}'$, which implies that

$$\left\|\nabla\mathcal{F}_u(\boldsymbol{w}) - \nabla\mathcal{F}_u\left(\boldsymbol{w}'\right)\right\| \le \beta\left\|\boldsymbol{w} - \boldsymbol{w}'\right\|, \forall u, \tag{25}$$

where β denotes the Lipschitz constant correlated with $\mathcal{F}(.)$.

Assumption 2: The stochastic gradients $\mathbf{g}_u(\boldsymbol{w})$ of $\mathcal{F}_u(\boldsymbol{w})$ are unbiased, with variance bounded by σ_u^2:

$$\mathbb{E}\left[\mathbf{g}_u(\boldsymbol{w})\right] = \nabla\mathcal{F}_u(\boldsymbol{w}), \forall\boldsymbol{w}, \forall u, \tag{26}$$

$$\mathbb{E}\left[\left\|\mathbf{g}_u(\boldsymbol{w}) - \nabla\mathcal{F}_u(\boldsymbol{w})\right\|^2\right] \le \sigma_u^2, \forall\boldsymbol{w}, \forall u, \tag{27}$$

Assumption 3: The local datasets' heterogeneity can be evaluated as

$$\mathbb{E}\left[\left\|\nabla\mathcal{F}_u(\boldsymbol{w}) - \nabla\mathcal{F}(\boldsymbol{w})\right\|^2\right] \le \Lambda_u^2, \forall\boldsymbol{w}, \forall u. \tag{28}$$

Theorem 1: Based on the analysis in [35], the negative impact caused by data heterogeneity can be mitigated when the devices have uniform transmission outage probabilities $q_u(p_u) = q, \forall u$. Based on this insight, we have Eq. (29), where

$$H_1 = \left(\frac{1}{2} - 2\eta\mathcal{E}\beta - \frac{4\eta^2\mathcal{E}^2\beta^2 + 4\eta^3\mathcal{E}^3\beta^3}{1 - 2\eta^2\mathcal{E}^2\beta^2}\right) \tag{30}$$

$$H_2 = \frac{\eta^2\mathcal{E}^2\beta^2 + \eta^3\mathcal{E}^3\beta^3}{1 - 2\eta^2\mathcal{E}^2\beta^2} \tag{31}$$

$$H_3 = \frac{1 + 2\eta^2\mathcal{E}^2\beta^2 + 4\eta^3\mathcal{E}^3\beta^3}{1 - 2\eta^2\mathcal{E}^2\beta^2} \tag{32}$$

with $\boldsymbol{w}^* = \arg\min_{\boldsymbol{w}}\mathcal{F}(\boldsymbol{w})$, $\chi = 1 - q_{\max}^U$, $q_{\max} = \max\{q_u\}$, and the learning rate η satisfies $0 < \eta < \frac{1}{4\beta}$.

Proof: See Appendix of Theorem 1 in [24]. ∎

According to Theorem 1, the convergence bound is influenced by the initial conditions $\left(\mathbb{E}[\mathcal{F}(\boldsymbol{w}^{(0)})] - \mathbb{E}[\mathcal{F}(\boldsymbol{w}^{(*)})]\right)$, gradient variance σ_u^2, data heterogeneity Λ_u^2, and the transmission outage probability q. A higher transmission outage probability q significantly hampers the training convergence. This is because only devices that do not experience transmission errors can participate in the training process. Therefore, a larger q means a lower probability of devices successfully participating in training, thereby requiring more training rounds to achieve convergence.

Substituting Eq. (29) into Eq. (33) leads to **Corollary 1**, which provides a lower bound on the number of training rounds required to achieve the target model convergence accuracy.

Corollary 1: The number of training rounds N for achieving target convergence accuracy ε, i.e., satisfying

$$\frac{1}{N}\sum_{n=0}^{N-1}\mathbb{E}\left[\left\|\nabla\mathcal{F}(\boldsymbol{w}_n)\right\|^2\right] \le \varepsilon, \tag{33}$$

can be given by

$$N \ge \frac{\Delta}{\left(\varepsilon - \bar{H}\right)\left(H_1\eta\mathcal{E}\right)} \tag{34}$$

where $\Delta = \mathbb{E}[\mathcal{F}(\boldsymbol{w}^{(0)})] - \mathbb{E}[\mathcal{F}(\boldsymbol{w}^{(*)})]$.

IV. Problem Formulation and Solution Algorithm

As previously mentioned, it is crucial to balance training performance and tranining time delay in the proposed SFL framework. Therefore, we formulate a training time delay minimization problem by jointly optimizing split layer $\{L_u\}$, transmission power $\{p_u\}$ and computing resources allocation $\{f_{s,u}\}$, which is given as

$$\mathcal{P}1: \min_{\{L_u\},\{f_{s,u}\},\{p_u\}} N\cdot\tau_{max} \tag{35a}$$

$$\text{s.t.} \quad \frac{1}{N}\sum_{n=0}^{N-1}\mathbb{E}\left[\left\|\nabla\mathcal{F}(\boldsymbol{w}_n)\right\|^2\right] \le \varepsilon, \tag{35b}$$

$$L_{\min} < L_u \le L_{\max}, \tag{35c}$$

$$L_u \in N^+, \tag{35d}$$

$$\mathbb{E}\left[\tau^n\left(\{L_u\},\{f_{s,u}\},\{p_u\}\right)\right] \le \tau_{max}, \tag{35e}$$

$$p_{\min} < p_u \le p_{\max}, \quad \forall u \in \mathcal{U}, \tag{35f}$$

$$\sum_{u=1}^{U} f_{s,u} \le f_{\max}. \tag{35g}$$

$$q_u(p_u) = q, \quad \forall u \in \mathcal{U} \tag{35h}$$

The objective function $N\cdot\tau_{max}$ is the upper bound of total training time, where N represent the minimum number of rounds required for the model to converge. Constraint (35b) guarantees the global convergence accuracy. Eqs. (35c) and (35d) define the constraints for split layer selection. Eq. (35e) specifies the upper bound on the expected delay per round. Eq. (35f) imposes constraints on the transmission power of each client. Eq. (35g) limits the maximum computing resources available to the server.

The optimization problem $\mathcal{P}1$ is challenging due to the coupling among $\{L_u\}$, $\{f_{s,u}\}$, and $\{p_u\}$. Moreover, the variables L_u are constrained to be integers, resulting in a more complex mixed-integer non-convex problem. To address this issue, the problem is decomposed into several subproblems, where the three decision variables are optimized in an alternating manner.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

Corollary 1 is first applied to explicitly express N, which is linearly proportional to the objective function. Hence, minimizing the objective function is equivalent to satisfying equality in constraint Eq. (34). Accordingly, the original problem $\mathcal{P}1$ is reformulated as

$$\mathcal{P}1': \min_{\{L_u\},\{f_{s,u}\},\{p_u\}} \Gamma(\{L_u\},\{f_{s,u}\},\{p_u\},\varepsilon), \tag{36a}$$

$$\text{s.t.} \quad constraints\ (35c)-(35h), \tag{36b}$$

where

$$\Gamma(\{L_u\},\{f_{s,u}\},\{p_u\},\varepsilon)=\frac{\Delta\cdot\tau_{max}}{\left(\varepsilon-\bar{H}\right)(H_1\eta\mathcal{E})} \tag{37}$$

A. Split Layer Selection With Fixed Transmission Power and Computing Resources

We consider the subproblem of $\mathcal{P}1$ that optimizes the split layer $\{L_u\}$, assuming fixed computing resources $\{f_{s,u}\}$ and transmission power $\{p_u\}$. Since the training rounds N is independent of $\{L_u\}$, given $\{p_u\}$ and $\{f_{s,u}\}$ satisfying constraints (35f) and (35g), $\mathcal{P}1$ reduces to

$$\mathcal{P}2: \min_{\{L_u\}} \quad \tau_{max} \tag{38a}$$

$$\text{s.t.} \quad \mathbb{E}\left[\tau_u^n(L_u)\right]\le\tau_{max}, \quad \forall u\in\mathcal{U}, n\in\mathcal{N}_\epsilon \tag{38b}$$

$$constraints \quad (35c)-(35d). \tag{38c}$$

Since p_u and $f_{s,u}$ are given as feasible, constraints (35f) and (35g) are automatically satisfied. Constraint (38b) is equivalent to constraint (35e). Owing to Eq. (14) and the substitution of Eqs. (16)–(22), constraint (38b) is simplified as follows:

$$\begin{aligned}&\frac{(1-q)\zeta_u(L_u)}{R_u^{\mathrm{DL}}}+\frac{(1-q)D_u\mathbf{C}_{c,u}^F(L_u)\cdot E}{f_u\varpi}\\&+\frac{(1-q)D_u\mathbf{C}_{c,u}^B(L_u)\cdot E}{f_u\varpi}+\frac{(1-q)D_u\psi_{a,u}(L_u)\cdot E}{R_u^{\mathrm{UL}}}\\&+\frac{(1-q)D_u\mathbf{C}_{s,u}(L_u)\cdot E}{f_{s,u}\varpi}+\frac{(1-q)D_u\psi_{g,u}(L_u)\cdot E}{R_u^{\mathrm{DL}}}\\&+\frac{(1-q)\zeta_u(L_u)}{R_u^{\mathrm{UL}}}\le\tau_{\max},\quad\forall u\in\mathcal{U}\end{aligned} \tag{39}$$

Since the split layer L_u is a discrete variable with a limited search space, the optimal split layer for each training round can be obtained via exhaustive search. For each device u, the objective value of problem $\mathcal{P}2$ is evaluated at every integer candidate split layer within $[L_{\min}, L_{\max}]$. The split layer that yields the minimum objective value is selected as the optimal configuration for that device. The detailed procedure is outlined in Algorithm IV-A

Algorithm 1 Split Layer Selection

1: **Input:**Computing resources allocation $\{f_{s,u}\}$, transmission power control $\{p_u\}$.
2: **Output:** $\{L_u^*\}$.
3: **for** device $u=1$ to U **do**
4: **for** split layer $l_u=1$ to L **do**
5: Calculate the value of $\tau(L_u)$, then find and record the split layer that corresponds to the smallest value of $\tau(L_u)$.
6: **end for**
7: **end for**

B. Computing Resources Optimization With Fixed Split Layer and Transmission Power

We consider the subproblem of $\mathcal{P}1$ that optimizes the computing resources allocation $\{f_{s,u}\}$, assuming fixed split layer selection $\{L_u\}$ and transmission power p_u. Since the training rounds N is independent of $\{f_{s,u}\}$, given $\{L_u\}$ and $\{p_u\}$ satisfying constraints (35c), (35d) and (35f), $\mathcal{P}1$ reduces to

$$\mathcal{P}3: \min_{\{f_{s,u}\}} \quad \tau_{max} \tag{40a}$$

$$\text{s.t.} \quad \mathbb{E}\left[\tau\left(\{f_{s,u}\}\right)\right]\le\tau_{max}, \tag{40b}$$

$$constraints \quad (35g). \tag{40c}$$

The U clients are sorted in ascending order of their computation delay $\tau_u(f_{s,u})$ as shown in (41), where $\hat{u}\in\mathcal{U}$ denotes the reordered client index.

$$\tau_0\le\tau_1\le\cdots\le\tau_{\hat{u}}\le\cdots\tau_{\max} \tag{41}$$

At the optimal solution of $\mathcal{P}3$, the server allocates computing resources such that all clients achieve the same training time τ_θ, i.e., Eq. (41) can be rewritten as

$$\tau_0=\tau_1=\cdots=\tau_{\hat{u}}=\tau_\theta. \tag{42}$$

Otherwise, if any $\tau_{\hat{u}}<\tau_\theta$, the server can reduce the computing resources allocated to client $\hat{u}$ and redistribute the surplus to others, thereby reducing the overall maximum delay $\tau_{\max}$. Therefore, problem $\mathcal{P}3$ can be reformulated as

$$\mathcal{P}3': \min_{\{f_{s,u}\}} \quad \tau_\theta \tag{43a}$$

$$\text{s.t.} \quad \mathbb{E}[\tau_u^n\left(f_{s,u}\right)]=\tau_\theta, \quad \forall u\in\mathcal{U}. \tag{43b}$$

$$constraints \quad (35g) \tag{43c}$$

$$\begin{aligned}\frac{\sum_{n=0}^{N-1}\mathbb{E}\left[\|\nabla\mathcal{F}(\mathbf{w}_n)\|^2\right]}{N}&\le\frac{1}{H_1\eta N\mathcal{E}}(\mathbb{E}[\mathcal{F}(\mathbf{w}^{(0)})]-\mathbb{E}[\mathcal{F}(\mathbf{w}^{(*)})])+(\frac{\eta\beta}{H_1U^2\chi(1-q)}+\frac{H_2}{H_1})\sum_u^U\frac{\sigma_u^2}{U}\\&+\frac{4\eta\mathcal{E}\beta}{H_1}\sum_{i=1}^U\frac{\Lambda_u^2}{U}+\frac{H_3}{H_1}\sum_{i=1}^U\frac{\Lambda_u^2}{U}+\frac{4\eta\mathcal{E}\beta}{H_1U(1-q)^{U-2}}\sum^U\Lambda_u^2\\&\triangleq\frac{1}{H_1\eta N\mathcal{E}}(\mathbb{E}[\mathcal{F}(\mathbf{w}^{(0)})]-\mathbb{E}[\mathcal{F}(\mathbf{w}^{(*)})])+\bar{H}\end{aligned} \tag{29}$$

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

Solving problem $\mathcal{P}3'$ yields the closed-form expression for the optimal computing resources allocated by the server to each client u as follows:

$$f_{\mathrm{s},u} = \frac{(1-q)D_u \mathbf{C}_{\mathrm{s},u} \cdot E}{\tau_\theta - (\tau^n_{\mathrm{dm},u} + \tau^n_{\mathrm{um},u} + \tau^n_{\mathrm{fp},u} + \tau^n_{\mathrm{sd},u} + \tau^n_{\mathrm{g},u} + \tau^n_{\mathrm{bp},u})} \tag{44}$$

Once τ_θ is known, the optimal resource allocation of the server can be obtained. In order to solve τ_θ, we substitute formula (44) into constraint (35g) and convert this inequality constraint into an equality constraint as

$$\sum_{u\in\mathcal{U}} \frac{(1-q)D_u \mathbf{C}_{\mathrm{s},u} \cdot E}{\tau_\theta - (\tau^n_{\mathrm{dm},u} + \tau^n_{\mathrm{um},u} + \tau^n_{\mathrm{fp},u} + \tau^n_{\mathrm{sd},u} + \tau^n_{\mathrm{g},u} + \tau^n_{\mathrm{bp},u})} = f_{\max} \tag{45}$$

Equation (44) can be shown to be strictly convex, ensuring the existence of a unique solution τ_θ for a given $f_{\max}$, which can be efficiently obtained using the CVX solver.

C. Transmission Power Optimization With Fixed Split Layer and Computing Resources

We consider a subproblem of $\mathcal{P}1$ that optimizes the transmission power $\{p_u\}$ while keeping the split layer selections $\{L_u\}$ and computing resource allocations $\{f_{\mathrm{s},u}\}$ fixed. Given that $\{L_u\}$ and $\{f_{\mathrm{s},u}\}$ satisfy constraints (35c), (35d), and (35g), the problem $\mathcal{P}1$ reduces to

$$\mathcal{P}4: \min_{\{p_u\}} \quad \Gamma(\{p_u\}, \varepsilon) \tag{46a}$$

$$\text{s.t. } \mathbb{E}\left[\tau\left(p_u\right)\right] \le \tau_{max}, \tag{46b}$$

$$constraints \quad (35b), (35f), (35h) \tag{46c}$$

It can be shown that the optimal solution to $\mathcal{P}4$ is achieved when all clients transmit data have the same transmission outage probability q. Under this condition, using Eq. (13), the transmission power for each client is given by

$$p_u = \frac{\Upsilon(WN_0 + I_u)}{h_u(q-1)}. \tag{47}$$

Substituting this result, the original problem $\mathcal{P}4$ can be reformulated as

$$\mathcal{P}4': \min_{q} \quad \Gamma(q, \varepsilon) \tag{48a}$$

$$\text{s.t.} \quad \mathbb{E}\left[\tau(q)\right] \le \tau_{max}, \tag{48b}$$

$$constraints \quad (35b), (35f), (35h), \tag{48c}$$

To efficiently solve $\mathcal{P}4'$, we design a Bayesian optimization-based algorithm [36] that utilizes the probability of improvement (PI) as the acquisition function to search for the near-optimal transmission outage probability q^*. The key idea is to iteratively construct a surrogate model for the objective function based on Gaussian processes, and then select the next evaluation point by maximizing the expected improvement. Let $f(q) = \Gamma(q, \varepsilon) = \frac{\Delta \cdot \mathbb{E}[\tau(q)]}{(\varepsilon - \bar{H})(H_1 \eta \mathcal{E})}$ denote the objective function of $\mathcal{P}4'$, and let $\mu(q)$ and $\sigma(q)$ denote the predicted mean and standard deviation of the surrogate model at point q. The probability of improvement (PI) is defined as:

$$\mathrm{PI}(q) = \Phi\left(\frac{f_{\mathrm{best}} - \mu(q)}{\sigma(q)}\right), \tag{49}$$

where f_{best} is the best observed objective value so far, and $\Phi(\cdot)$ is the cumulative distribution function of the standard normal distribution.

At each iteration, the algorithm selects the next query point q^{next} by solving:

$$q^{\mathrm{next}} = \arg\max_{q\in\mathcal{Q}} \mathrm{PI}(q), \tag{50}$$

where $\mathcal{Q}$ denotes the feasible search space for q, typically defined by system constraints and hardware limitations.

The procedure is repeated until a stopping criterion is met, such as a maximum number of iterations or convergence of the objective value. The overall algorithm ensures a balance between exploration and exploitation, and significantly reduces the number of evaluations compared to brute-force methods.

After getting the optimal transmission outage probability q^*, we can obtain the optimal transmission power $\{p_u^*\}$ accordingly by Eq. (47).

D. Joint Optimization on Split Layer Selection, Computing Resources Allocation and Transmission Power Control

This section presents a three-stage joint scheduling algorithm by integrating the previously derived optimal solutions for split layer selection, computing resource allocation, and transmission power control. At iteration k, the optimal split layers $\{(L_u)_k^*\}$ are determined as described in Section III.A, given the fixed computing resource allocation $\{(f_{\mathrm{s},u})_{k-1}^*\}$ and transmission power $\{(p_u)_{k-1}^*\}$ for all clients u. Subsequently, the optimal computing resource allocation $\{(f_{\mathrm{s},u})_k^*\}$ is obtained by solving problem $\mathcal{P}3'$ using the updated split layers $\{(L_u)_k^*\}$ and fixed transmission power $\{(p_u)_{k-1}^*\}$. Then, with the updated resource allocation $\{(f_{\mathrm{s},u})_k^*\}$ and split layers $\{(L_u)_k^*\}$, the optimal power control $\{(p_u)_k^*\}$ is computed by solving problem $\mathcal{P}4'$. In the next iteration, the split layers $\{(L_u)_k^*\}$ are updated based on the latest resource allocation $\{(f_{\mathrm{s},u})_k^*\}$ and power control $\{(p_u)_k^*\}$. The resource allocation and power control are similarly updated using the most recent values of the other variables. This iterative process continues until convergence is achieved, defined by

$$\tau\left((L_u)_{k+1}^*, (f_u)_{k+1}^*, (p_u)_{k+1}^*\right) - \tau\left((L_u)_k^*, (f_u)_k^*, (p_u)_k^*\right) \le \varrho, \tag{51}$$

where ϱ is a manually set convergence threshold. The overall algorithmic workflow is presented in Algorithm 2.

V. Experiment Results

A. Experiment Settings

In the following experiments, we evaluate our method on two widely used datasets: CIFAR-10 and MNIST. The CIFAR-10 dataset contains 60,000 color images (32 × 32 pixels) in 10 classes, with 50,000 images used for training and 10,000 for testing [37]. The MNIST dataset comprises 70,000

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Joint Optimization Algorithm

Require: ε, $L^{\min}$, $L^{\max}$, $p^{\min}$, $p^{\max}$, $f^{\max}$
Ensure: $\{(L_u^n)_k^*\}$, $\{(p_u^n)\}_k^*$, $\{(f_{s,u}^n)_k^*\}$
while gap $\geq \varrho$ **do**
Obtain $\{(L_u^n)_k^*\}$ by solving $\mathcal{P}2$ using $\{(p_u^n)_{k-1}^*\}$ and $\{(f_{s,u}^n)_{k-1}^*\}$.
Obtain $\{(f_{s,u}^n)_k^*\}$ by solving $\mathcal{P}3'$ using $\{(p_u^n)_{k-1}^*\}$ and $(L_u^n)_k^*$.
Obtain $\{(p_u^n)_k^*\}$ by solving $\mathcal{P}4$ using $\{(f_{s,u}^n)_k^*\}$ and $(L_u^n)_k^*$.
Calculate the gap defined in Eq. (51).
$k = k + 1$
end while

TABLE I
PARAMETER SETTINGS

Parameter	Value	Parameter	Value
$p^{\max}$	0.1W	$p^{\min}$	0.01W
W_{BS}	10MHz	ς_u	0.015
p_{B}	1W	$f_{\max}$	300GFLOPs
I_u^n	$\text{U}[10^{-8}, 2\times10^{-8}]$	d_u^n	$\text{U}[100, 300]m$
N_0	-174 dbm/Hz	k	1.25×10^{-26}
Υ	0.023dB		

grayscale images (28 × 28 pixels) of handwritten digits ranging from 0 to 9. We use the standard split of 60,000 for training and 10,000 for testing [38]. We adopt the ResNet-18 architecture for image classification. The model begins with a 7 × 7 convolution and a max-pooling layer, followed by four groups of residual blocks. Each block incorporates residual connections to ensure efficient gradient flow and stable training. As the network deepens, it gradually increases channel depth while reducing spatial resolution, concluding with a global average pooling layer. A final fully connected layer outputs the predicted class label.

In our simulation experiments, we set the total number of training rounds to 1000 and deploy $U = 10$ devices. Two types of data distribution are considered. For the i.i.d. setting, each device is assigned 4000 samples uniformly drawn from the global dataset. For the non-i.i.d. setting, each device is allocated between 2000 and 4000 samples following a Dirichlet distribution with a concentration parameter of 0.9 to simulate statistical heterogeneity. The key experimental parameters are summarized in Table I.

B. Performance Comparison of Different Architectures

In this subsection, we evaluate the performance of the proposed scheme by comparing it with several representative learning architectures. The comparison includes the following schemes:

- **CL (Centralized Learning):** All training data is uploaded to a central server, where the entire model is trained without any device-side computation. This serves as an upper-bound benchmark in terms of accuracy.
- **FL (Federated Learning):** The classic FedAvg algorithm is employed. Each device trains a full local model using its own data and periodically uploads model weights for aggregation at the server.
- **SL (Split Learning):** The model is split between client and server at a fixed split layer. Each client performs forward propagation and then sends intermediate features to the server sequentially. The server completes backward propagation and returns gradients one by one.
- **SFL($L = k$):** The Selective Split Federated Learning framework, where each client independently chooses a split layer $L_u = k$ and participates in collaborative training. Unlike SL, all clients conduct local computation in parallel and communicate with the server simultaneously. We evaluate SFL under different split layer indices $k = 2, 4, 6$.
- **Proposed** Our proposed framework builds upon SFL by introducing joint optimization strategies, including adaptive split layer selection, resource allocation, and transmission power control

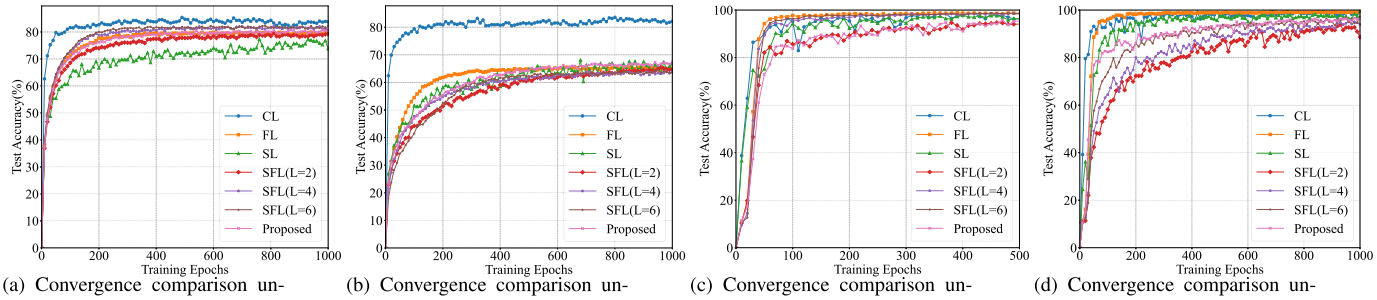
We conduct comprehensive experiments on both CIFAR-10 and MNIST datasets, under both i.i.d. and non-i.i.d. settings. As illustrated in Fig. 2, CL achieves the fastest convergence and highest accuracy across all scenarios, owing to centralized data and complete model visibility. In contrast, FL and SFL converge more slowly due to decentralized data distribution and limited global information visibility. Notably, SFL exhibits convergence behavior comparable to that of FL in all four scenarios, with only marginal differences in convergence speed and final accuracy. This indicates that SFL can be a practical alternative to traditional FL, especially when privacy requirements or system limitations hinder full-model training.

Fig. 3 shows the overall training latency for different schemes. CL exhibits the lowest training delay across all scenarios. This is because our experiments are conducted on CIFAR-10 and MNIST datasets using ResNet-18, where the size of raw data is significantly smaller than that of model parameters. As a result, CL—which only transmits raw data without requiring iterative model exchanges—incurs less communication overhead. In contrast, FL introduces additional latency due to periodic transmission of full model parameters. SFL further reduces the delay by transmitting only partial model updates after the feature-splitting process, and by offloading part of the computation to the server, which reduces the local burden on clients. SL, on the other hand, suffers from the highest latency. This is primarily attributed to its serialized client-server communication process, where each client must wait for the previous one to complete its transmission and computation before proceeding—resulting in poor parallelism and inefficient resource utilization. Our proposed scheme achieves the lowest latency among all SFL-based methods. However, the margin of improvement is relatively small in this idealized setup, likely because the experiments are conducted under ideal channel conditions without considering transmission errors or packet loss.

C. Impact of the Split Layer Selection

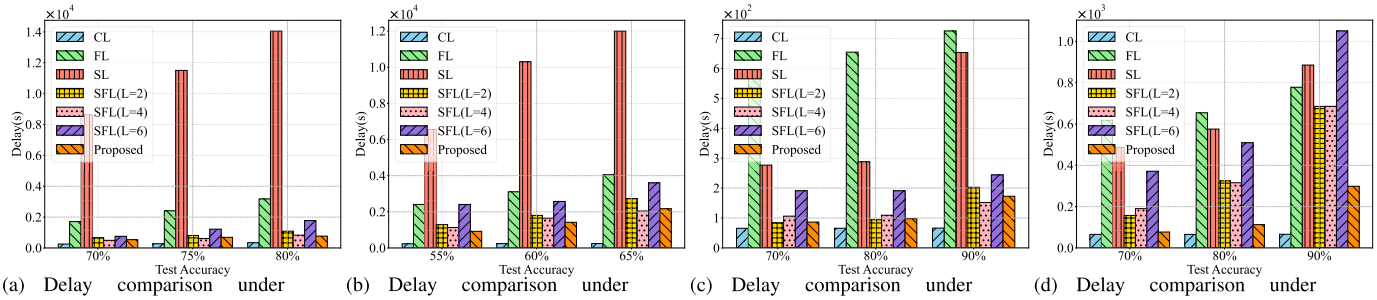
We conducted experiments to measure the training delay per round at different split layers in ResNet18, as illustrated

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.



(a) Convergence comparison under CIFAR-10 dataset (i.i.d. setting). (b) Convergence comparison under CIFAR-10 dataset (non-i.i.d. setting). (c) Convergence comparison under MNIST dataset (i.i.d. setting). (d) Convergence comparison under MNIST dataset (non-i.i.d. setting).

Fig. 2. Convergence comparison of different architecture.



(a) Delay comparison under CIFAR-10 dataset (i.i.d. setting). (b) Delay comparison under CIFAR-10 dataset (non-i.i.d. setting). (c) Delay comparison under MNIST dataset (i.i.d. setting). (d) Delay comparison under MNIST dataset (non-i.i.d. setting).

Fig. 3. Delay comparison of different architecture.

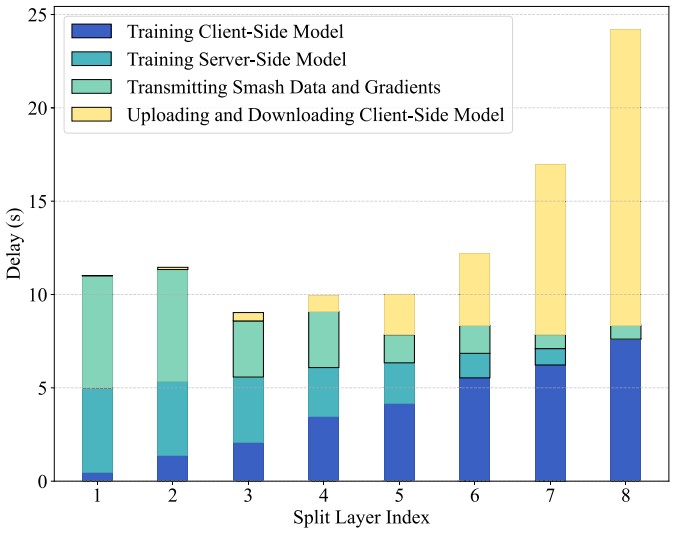


Fig. 4. Delay comparison of different split layer.

in Fig. 4. The experimental settings are consistent with those listed in Table I. As shown in the figure, the training delay for early split layers is noticeably higher than that for middle layers. This is attributed to the larger volume of smash data generated at lower-layer splits, leading to increased uplink transmission delay and server-to-client gradient feedback delay. Consequently, the total training delay per round increases.

Similarly, the training delay at later split layers is also higher than that of middle layers. This is because a higher split layer implies that more model layers are trained locally on the client device, where computational resources are significantly limited compared to the server. As a result, the client-side computation becomes the bottleneck, increasing the overall training delay per round.

Evidently, the optimal selection of the split layer depends on factors such as the allocated server-side computing resources and the data volume per training round. Therefore, it is necessary to determine the optimal split layer via the proposed optimization algorithm, tailored to specific task configurations, to minimize the total training delay.

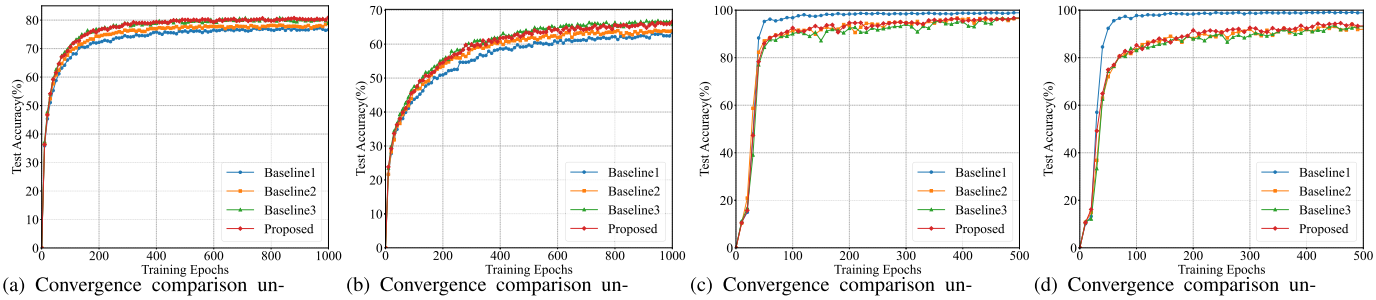
D. Impact of Optimization Algorithm

In this subsection, we evaluate the performance of the proposed scheme with the following baselines:

- **Baseline 1:** Jointly optimizes the transmission power p_u and computing resource allocation $f_{s,u}$, while the split layer L_u is fixed to the same value across all clients.
- **Baseline 2:** Jointly optimizes the split layer L_u and computing resource allocation $f_{s,u}$, while all clients transmit with a fixed and identical power p_u.
- **Baseline 3:** Jointly optimizes the split layer L_u and transmission power p_u, while the computing resources $f_{s,u}$ are evenly divided among all clients from the total server-side capacity.
- **Proposed:** The proposed optimization framework that jointly optimizes transmission power p_u, computing resource allocation $f_{s,u}$, and split layer selection L_u for each client.

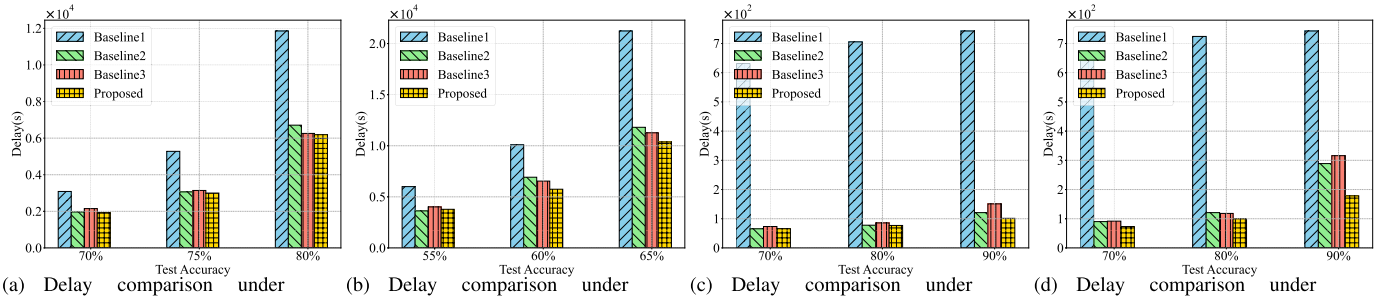
We conduct comprehensive experiments on both CIFAR-10 and MNIST datasets, under both i.i.d. and non-i.i.d. settings.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.



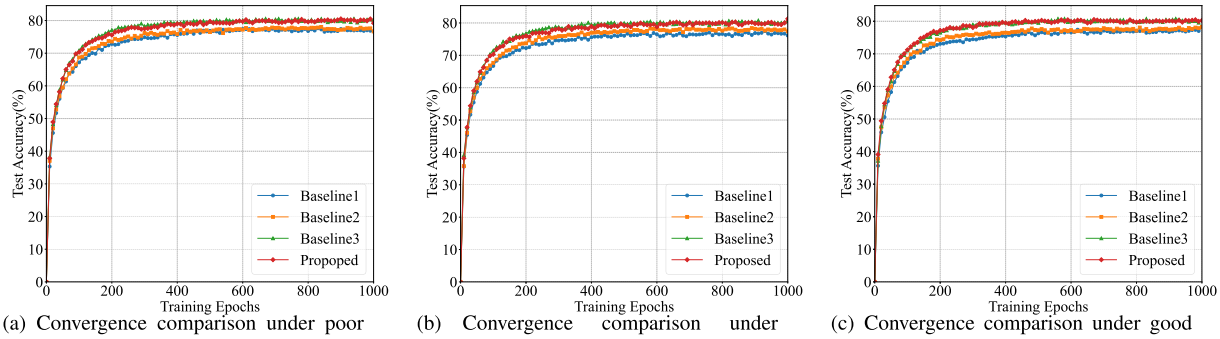
(a) Convergence comparison under CIFAR-10 dataset (i.i.d. setting).
(b) Convergence comparison under CIFAR-10 dataset (non-i.i.d. setting).
(c) Convergence comparison under MNIST dataset (i.i.d. setting).
(d) Convergence comparison under MNIST dataset (non-i.i.d. setting).

Fig. 5. Convergence comparison of different optimization algorithm.



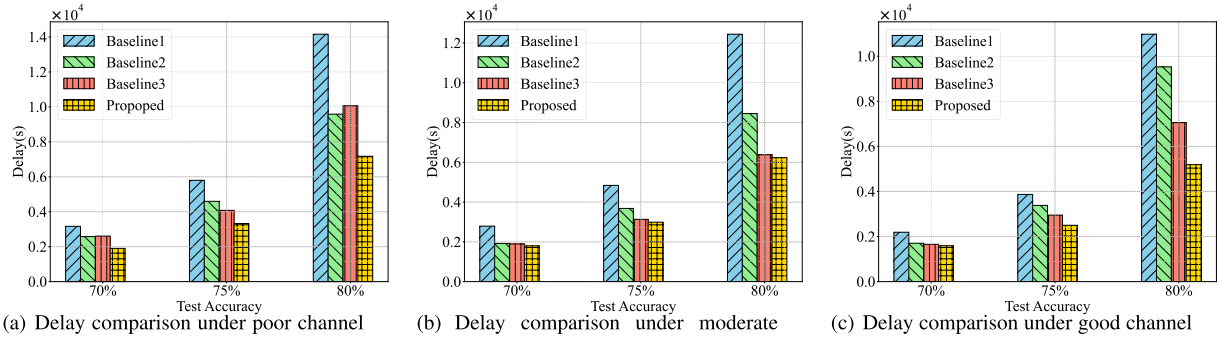
(a) Delay comparison under CIFAR-10 dataset (i.i.d. setting).
(b) Delay comparison under CIFAR-10 dataset (non-i.i.d. setting).
(c) Delay comparison under MNIST dataset with (i.i.d. setting).
(d) Delay comparison under MNIST dataset with (non-i.i.d. setting).

Fig. 6. Delay comparison of different optimization algorithm.



(a) Convergence comparison under poor channel conditions (CIFAR-10 dataset, i.i.d. setting).
(b) Convergence comparison under moderate channel conditions (CIFAR-10 dataset, i.i.d. setting).
(c) Convergence comparison under good channel conditions (CIFAR-10 dataset, i.i.d. setting).

Fig. 7. Convergence Comparison of different optimization algorithm under different channels.



(a) Delay comparison under poor channel conditions (CIFAR-10 dataset, i.i.d. setting).
(b) Delay comparison under moderate channel conditions (CIFAR-10 dataset, i.i.d. setting).
(c) Delay comparison under good channel conditions (CIFAR-10 dataset, i.i.d. setting).

Fig. 8. Delay Comparison of different optimization algorithm under different channels.

Fig. 5 illustrates the convergence differences among various baselines, while Fig. 6 shows the overall training delay required to reach the same accuracy. **Baseline 1** achieves slightly faster convergence compared to the other baselines,

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

but suffers from the highest overall delay. This might be because the fixed split layer L_u reduces the variability in model updates across devices, leading to stable convergence. However, as shown in Fig. 4, the choice of L_u significantly affects per-round delay. The cumulative effect results in the longest total training time. **Baseline 2** shows the slowest convergence among all baselines. The lack of transmission power optimization (p_u) leads to poor adaptation to varying channel conditions, resulting in more frequent device dropouts and increased number of training rounds required for convergence. **Baseline 3** achieves moderate convergence speed and lower delay than Baseline 1 and Baseline 2. This improvement is attributed to the optimization of L_u and p_u, which reduces communication cost. However, the absence of computing resource allocation ($f_{s,u}$) results in imbalanced processing times among heterogeneous devices. Since the per-round delay is determined by the slowest device, the total delay increases accordingly. In contrast, the **proposed** scheme consistently achieves the best performance in terms of training delay while maintaining competitive convergence behavior. Through the joint optimization of split layer L_u, transmission power p_u, and computing resource allocation $f_{s,u}$, it achieves a balanced trade-off between resource efficiency and learning performance. This coordination harmonizes computation and communication workloads across heterogeneous devices, effectively minimizing per-round latency without significantly compromising convergence.

E. Impact of Channel Quality

In this subsection, we evaluate how varying wireless channel conditions influence the convergence performance and the overall delay of the proposed scheme compared with baseline methods. The baseline configurations remain consistent with those described in Subsection D. Fig. 8 illustrates the test accuracy trajectories under three Rayleigh fading scenarios, characterized by different fading coefficients ς: poor channel quality ($\varsigma = 0.01$), moderate channel quality ($\varsigma = 0.02$), and good channel quality ($\varsigma = 0.03$). These coefficients directly affect transmission outage probabilities, thereby impacting the number of effective devices involved in each training round. As shown in the Fig. 8, when the channel quality is good ($\varsigma = 0.03$), the impact of device dropouts is relatively minor, and both baseline2 and the proposed method achieve comparable convergence rates. However, under poor channel conditions ($\varsigma = 0.01$), the lack of transmission power optimization in baseline2 results in a significantly higher outage probability, leading to frequent device dropouts and substantially slower convergence. In contrast, our proposed scheme maintains robust performance across all channel conditions, consistently achieving faster convergence and lower global training delay. This demonstrates the effectiveness and adaptability of our joint optimization strategy, particularly the dynamic adjustment of transmission power, in mitigating the adverse effects of unreliable wireless links.

VI. Conclusion

This paper investigates the deployment challenges of Split Federated Learning (SFL) in resource-constrained edge computing networks characterized by device heterogeneity and unreliable wireless channels. To address the trade-offs among convergence speed, communication reliability, and resource constraints, we formulate a joint optimization problem that minimizes the total training latency required to achieve a target model accuracy. The proposed framework jointly optimizes split layer selection, computing resource allocation, and transmission power control. An efficient Block Coordinate Descent (BCD)-based algorithm is developed to solve the problem. Extensive experiments validate the effectiveness of the proposed method, demonstrating significant reductions in training latency and improved adaptability in unreliable edge network environments.

References

[1] C. Xu et al., "Energy consumption and time-delay optimization of dependency-aware tasks offloading for Industry 5.0 applications," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 1590–1600, Feb. 2024.

[2] Q. Lin, S. Jiang, Z. Zhen, T. Chen, C. Wei, and H. Lin, "Fed-PEMC: A privacy-enhanced federated deep learning algorithm for consumer electronics in mobile edge computing," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4073–4086, Feb. 2024.

[3] J. Pei, W. Liu, J. Li, L. Wang, and C. Liu, "A review of federated learning methods in heterogeneous scenarios," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 5983–5999, Aug. 2024.

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017.

[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[6] S. T. Ahmed, V. V. Kumar, and J. Jeong, "Heterogeneous workload-based consumer resource recommendation model for smart cities: Ehealth edge-cloud connectivity using federated split learning," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4187–4196, Feb. 2024.

[7] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *J. Netw. Comput. Appl.*, vol. 116, pp. 1–8, Aug. 2018.

[8] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, Aug. 2022, pp. 8485–8493.

[9] G. Anbazhagan, M. Maragatharajan, and T.-H. Kim, "Smart battery management system for mobile consumer electronics based on federal split learning model," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 5878–5886, Aug. 2024.

[10] X. Liu, Y. Deng, and T. Mahmoodi, "Wireless distributed learning: A new hybrid split and federated learning approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 4, pp. 2650–2665, Apr. 2023.

[11] M. Zhang, L. Qu, P. Singh, J. Kalpathy-Cramer, and D. L. Rubin, "SplitAVG: A heterogeneity-aware federated deep learning method for medical imaging," *IEEE J. Biomed. Health Informat.*, vol. 26, no. 9, pp. 4635–4644, Sep. 2022.

[12] Z. Yang et al., "Dynamic corrected split federated learning with homomorphic encryption for U-shaped medical image networks," *IEEE J. Biomed. Health Informat.*, vol. 27, no. 12, pp. 5946–5957, Dec. 2023.

[13] D. Zhang, J. Le, N. Mu, G. Liu, and T. Xiang, "Privacy-preserving federated learning based on dataset condensation," *IEEE Trans. Consum. Electron.*, vol. 71, no. 1, pp. 748–760, Feb. 2025.

[14] L. Zhang, J. Xu, P. Vijayakumar, P. K. Sharma, and U. Ghosh, "Homomorphic encryption-based privacy-preserving federated learning in IoT-enabled healthcare system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2864–2880, Sep. 2023.

[15] C. Xu, J. Li, Y. Liu, Y. Ling, and M. Wen, "Accelerating split federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 6, pp. 5587–5599, Jun. 2024.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.

[16] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, Jan. 2020.

[17] W. Wu et al., "Split learning over wireless networks: Parallel design and resource management," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1051–1066, Apr. 2023.

[18] Z. Lin, G. Qu, W. Wei, X. Chen, and K. K. Leung, "AdaptSFL: Adaptive split federated learning in resource-constrained edge networks," 2024, *arXiv:2403.13101*.

[19] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Comput. Archit. News*, vol. 45, no. 1, pp. 615–629, 2017.

[20] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive DNN surgery for inference acceleration on the edge," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, Paris, France, Apr. 2019, pp. 1423–1431.

[21] S. Wang, X. Zhang, H. Uchiyama, and H. Matsuda, "HiveMind: Towards cellular native machine learning model splitting," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 2, pp. 626–640, Feb. 2022.

[22] Z. Lin, Z. Chen, X. Chen, W. Ni, and Y. Gao, "HASFL: Heterogeneity-aware split federated learning over edge computing systems," 2025, *arXiv:2506.08426*.

[23] L. Li et al., "Energy and spectrum efficient federated learning via high-precision over-the-air computation," 2022, *arXiv:2208.07237*.

[24] Y. Tang, G. Zhu, W. Xu, M. H. Cheung, T.-M. Lok, and S. Cui, "Integrated sensing, computation, and communication for UAV-assisted federated edge learning," *IEEE Trans. Wireless Commun.*, vol. 24, no. 4, pp. 2647–2662, Apr. 2025.

[25] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 269–283, Jan. 2021.

[26] M. Salehi and E. Hossain, "Federated learning in unreliable and resource-constrained cellular wireless networks," *IEEE Trans. Commun.*, vol. 69, no. 8, pp. 5136–5151, Aug. 2021.

[27] G. Zhu, Y. Du, D. Gündüz, and K. Huang, "One-bit over-the-air aggregation for communication-efficient federated edge learning: Design and convergence analysis," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 2120–2135, Mar. 2021.

[28] V. Stephanie, I. Khalil, and M. Atiquzzaman, "DSFL: A decentralized SplitFed learning approach for healthcare consumers in the metaverse," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2107–2115, Feb. 2024.

[29] M. Chen, H. V. Poor, W. Saad, and S. Cui, "Convergence time optimization for federated learning over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 4, pp. 2457–2471, Apr. 2021.

[30] J. Wang, L. Bai, Z. Fang, R. Han, J. Wang, and J. Choi, "Age of information based URLLC transmission for UAVs on pylon turn," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 8797–8809, Jun. 2024.

[31] Z. Zhou et al., "Secure and latency-aware digital twin assisted resource scheduling for 5G edge computing-empowered distribution grids," *IEEE Trans. Ind. Informat.*, vol. 18, no. 7, pp. 4933–4943, Jul. 2022.

[32] B. Xu, H. Zhao, H. Cao, S. Garg, and M. Alrashoud, "Long-term energy management empowered hierarchical federated learning for smart consumer electronics," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 1155–1166, Feb. 2024.

[33] X. Fan, Y. Wang, Y. Huo, and Z. Tian, "1-bit compressive sensing for efficient federated learning over the air," *IEEE Trans. Wireless Commun.*, vol. 22, no. 3, pp. 2139–2155, Mar. 2023.

[34] J. Zhang, X. Li, K. Gu, W. Liang, and K. Li, "Secure aggregation in heterogeneous federated learning for digital ecosystems," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 1995–2003, Feb. 2024.

[35] Y. Wang, Y. Xu, Q. Shi, and T.-H. Chang, "Quantized federated learning under transmission delay and outage constraints," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 323–341, Jan. 2022.

[36] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, Jan. 2016.

[37] A. Krizhevsky, "Learning multiple layers of features from tiny images," Master thesis, Univ. Toronto, Toronto, ON, Canada, 2009.

[38] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.



Xianghe Wang received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 2024, where he is currently pursuing the Ph.D. degree with the Department of Electronic Engineering. His research interests mainly focus on edge intelligence, federated learning, wireless AI, and UAV/AUV networks.



Shaoyang Song received the bachelor's degree in communication engineering from Beijing Institute of Technology in 2023, where he is currently pursuing the master's degree with the School of Information and Electronics. His current research interests include digital twin and electromagnetic environment modeling, machine learning, scene recognition, and federated learning.



Zekai Zhang received the B.S. degree from North University of China in 2021, and the M.S. degree in electronic and information engineering from Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, China, in 2024, where he is currently pursuing the Ph.D. degree. His research interests include federated learning, multi-agent reinforcement learning, integrated sensing and communications (ISAC), and generative artificial intelligence and network optimization.

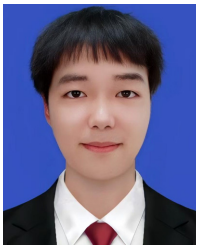


Xiangwang Hou (Member, IEEE) received the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2025. He is currently a Post-Doctoral Researcher (Shuimu Tsinghua Scholar Program) with the Department of Electronic Engineering, Tsinghua University. From 2023 to 2024, he was a Joint Ph.D. Student with the School of Computer Science and Engineering, Nanyang Technological University, Singapore, under the supervision of Prof. Dusit Niyato. From 2020 to 2021, he was an Algorithm Engineer with Huawei Technologies Company Ltd., and Tsinghua University. His research interests include edge intelligence, federated learning, wireless AI, and UAV/AUV networks. He was a recipient of the Best Paper Award of IEEE ICC.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.



Zhiying Li received the B.S. degree from the North University of China, Taiyuan, China, in 2021, and the M.S. degree from Beihang University in 2024. His research interests include edge-assisted federated learning and multi-agent collaboration in industrial IoT.



Tianyu Xing received the B.S. degree in mechanical engineering from Zhengzhou University, Zhengzhou, China, in 2019, and the M.S. degree in electronic and information engineering from Tsinghua University, Beijing, China, in 2024. His research interests include federated learning, wireless networks, and deep reinforcement learning for multi-agent systems.



Xiao-Ping Zhang (Fellow, IEEE) Xiao-Ping Zhang received B.S. and Ph.D. degrees from Tsinghua University, in 1992 and 1996, respectively, both in Electronic Engineering. He holds an MBA in Finance, Economics and Entrepreneurship with Honors from the University of Chicago Booth School of Business, Chicago, IL.

He is Penrui Chair Professor at Tsinghua Shenzhen International Graduate School (SIGS), Tsinghua University. He was the founding Dean of Institute of Data and Information (iDI) at Tsinghua SIGS and Chair Professor at Tsinghua-Berkeley Shenzhen Institute (TBSI). He had been with the Department of Electrical, Computer and Biomedical Engineering, Toronto Metropolitan University (Formerly Ryerson University), Toronto, ON, Canada, as a Professor and the Director of the Communication and Signal Processing Applications Laboratory (CASPAL), and has served as the Program Director of Graduate Studies. His research interests include sensor networks and IoT, machine learning/AI/robotics, statistical signal processing, image and multimedia content analysis, and applications in big data, finance, and marketing.

Dr. Zhang is Fellow of the Canadian Academy of Engineering, Fellow of the Engineering Institute of Canada, Fellow of the IEEE, a registered Professional Engineer in Ontario, Canada, and a member of Beta Gamma Sigma Honor Society. He is the general Co-Chair for the IEEE International Conference on Acoustics, Speech, and Signal Processing, 2021. He is the general co-chair for 2017 GlobalSIP Symposium on Signal and Information Processing for Finance and Business, and the general co-chair for 2019 GlobalSIP Symposium on Signal, Information Processing and AI for Finance and Business. He was an elected Member of the ICME steering committee. He is the general chair for ICME2024 and BioCAS2023. He is Editor-in-Chief for the IEEE JOURNAL OF SELECTED TOPICS IN SIGNAL PROCESSING. He was Senior Area Editor for the IEEE TRANSACTIONS ON IMAGE PROCESSING and the IEEE TRANSACTIONS ON SIGNAL PROCESSING. He was Associate Editor for the IEEE TRANSACTIONS ON IMAGE PROCESSING, the IEEE Transactions on Multimedia, the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY, the IEEE TRANSACTIONS ON SIGNAL PROCESSING, and the IEEE SIGNAL PROCESSING LETTERS. He was selected as IEEE Distinguished Lecturer by the IEEE Signal Processing Society and by the IEEE Circuits and Systems Society.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:41:11 UTC from IEEE Xplore. Restrictions apply.